

DATA VISUALIZATION

Plots, Visualization, Storytelling

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-III



UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

- 1 Data Visualization
- 2 Visual Storytelling and Dashboard Design



Outline

- 1 Data Visualization
- 2 Visual Storytelling and Dashboard Design



Why Data Visualization?

Anscombe's Quartet (1973)

Four datasets with **identical** summary statistics ($\bar{x}$, $\bar{y}$, s_x , s_y , r) but **completely different** visual patterns. Statistics alone can **deceive**; visualization **reveals truth**.

Visualization enables:

- **Pattern recognition** faster than any table
- **Outlier detection** at a glance
- Communication of insights to **non-technical** audiences
- Monitoring of **data distributions** over time

Python ecosystem:

- **matplotlib**: low-level publication quality
- **seaborn**: statistical defaults on matplotlib
- **plotly**: interactive web-ready charts
- **bokeh**: real-time streaming dashboards



Why Data Visualization?

Anscombe's Quartet (1973)

Four datasets with **identical** summary statistics ($\bar{x}$, $\bar{y}$, s_x , s_y , r) but **completely different** visual patterns. Statistics alone can **deceive**; visualization **reveals truth**.

Visualization enables:

- **Pattern recognition** faster than any table
- **Outlier detection** at a glance
- Communication of insights to **non-technical** audiences
- Monitoring of **data distributions** over time

Python ecosystem:

- matplotlib: low-level **publication** quality
- seaborn: statistical **defaults** on matplotlib
- plotly: **interactive** web-ready charts
- bokeh: real-time **streaming** dashboards

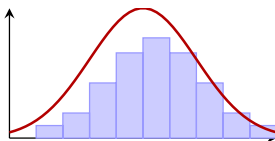


Types of Plots I: *Distribution* and *Comparison*

Distribution

- **Histogram:** shape, modality, **skewness** of a continuous variable; bin width critical
- **KDE:** smooth **density** estimate; area = 1
- **Box Plot:** median, IQR, **outliers** at a glance

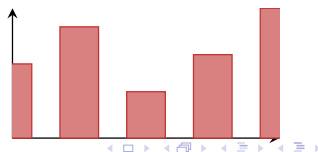
KDE + Histogram



Comparison

- **Bar Chart:** aggregate values per **category**; sorted for readability
- **Grouped Bar:** **multiple series** per category
- **Radar:** multi-axis comparison — use **sparingly**

Bar Chart



Types of Plots II: *Relationship and Composition*

Relationship

- **Scatter**: two continuous vars; add **regression** line to show trend
- **Bubble**: scatter + **size** encoding a 3rd variable
- **Heatmap**: pairwise **correlation** matrix or 2-D aggregates
- **Pair Plot**: all pairwise scatter plots — EDA for multivariate data

Composition & Time

- **Pie/Donut**: part-to-whole; use only with **few categories** (≤ 5)
- **Stacked Bar**: composition across categories
- **Line Plot**: **trend over time**; best for time-series data
- **Area Chart**: stacked lines encoding **cumulative volumes**



Types of Plots II: *Relationship and Composition*

Relationship

- **Scatter**: two continuous vars; add **regression** line to show trend
- **Bubble**: scatter + **size** encoding a 3rd variable
- **Heatmap**: pairwise **correlation** matrix or 2-D aggregates
- **Pair Plot**: all pairwise scatter plots — EDA for multivariate data

Composition & Time

- **Pie/Donut**: part-to-whole; use only with **few categories** (≤ 5)
- **Stacked Bar**: composition **across categories**
- **Line Plot**: **trend over time**; best for time-series data
- **Area Chart**: stacked lines encoding **cumulative volumes**



Histogram: Understanding Data Distribution

When to Use

Single continuous variable; understand **shape** (symmetric, skewed, bimodal); detect **outliers** and identify **gaps**.

Key Parameters:

- **Bin width**: critical—too small → noise; too large → loss of detail
- **Sturges rule**: $k = \lceil \sqrt{n} \rceil$ bins
- **Normalize**: compare different-sized datasets

Python Example:

```
sns.histplot(data, bins=30,
kde=True)
```



Interpretation Guide:

- **Bell curve**: symmetric, normal distribution
- **Left skew**: tail left, median > mean
- **Right skew**: tail right, mean > median
- **Bimodal**: two peaks → two populations mixed

Common Pitfall:

- Bin width heavily influences



Histogram: Understanding Data Distribution

When to Use

Single continuous variable; understand **shape** (symmetric, skewed, bimodal); detect **outliers** and identify **gaps**.

Key Parameters:

- **Bin width**: critical—too small → noise; too large → loss of detail
- **Sturges rule**: $k = \lceil \sqrt{n} \rceil$ bins
- **Normalize**: compare different-sized datasets

Python Example:

```
sns.histplot(data, bins=30,  
kde=True)
```



Interpretation Guide:

- **Bell curve**: symmetric, normal distribution
- **Left skew**: tail left, median > mean
- **Right skew**: tail right, mean > median
- **Bimodal**: two peaks → two populations mixed

Common Pitfall:

- Bin width heavily influences



KDE and Box Plot: Distribution Summary

KDE: Smooth Density Estimate

When: smooth probability;
compare multiple distributions

Interpretation:

- Area under curve = 1 (total probability)
- Peak height $\approx$ mode (most common value)
- Width $\approx$ spread/dispersion

Code: `sns.kdeplot(data,
fill=True)`

Box Plot: Quartiles at a Glance

When: detect outliers, compare
groups, handle skew

Components:

- **Box:** Q1–Q3 (50%)
- **Line in box:** median (Q2)
- **Whiskers:** $\pm 1.5 \times \text{IQR}$
- **Dots:** outliers beyond whiskers

Code: `sns.boxplot(data=df,
x='group', y='value')`



KDE and Box Plot: Distribution Summary

KDE: Smooth Density Estimate

When: smooth probability;
compare multiple distributions

Interpretation:

- Area under curve = 1 (total probability)
- Peak height $\approx$ mode (most common value)
- Width $\approx$ spread/dispersion

Code: `sns.kdeplot(data,
fill=True)`

Box Plot: Quartiles at a Glance

When: detect outliers, compare
groups, handle skew

Components:

- **Box:** Q1–Q3 (50%)
- **Line in box:** median (Q2)
- **Whiskers:** $\pm 1.5 \times \text{IQR}$
- **Dots:** outliers beyond whiskers

Code: `sns.boxplot(data=df,
x='group', y='value')`



Bar Chart: Categorical Aggregation I

When to Use

Categorical variable with aggregated numeric values (*sum*, *mean*, *count*); compare **across categories**; **sorted order** critical for readability.

Best Practices:

- **Sort bars** by value (descending) for clarity
- **Label axes** with units (e.g., Revenue (\$M))
- Use **one color** unless hierarchy exists
- Avoid 3D effects; they **distort** perception
- Keep categories < 10 ; rotate labels if needed

```
sns.barplot(data=df, x='region', y='sales')
```



Bar Chart: Categorical Aggregation II

When to Use

Categorical variable with aggregated numeric values (*sum*, *mean*, *count*); compare **across categories**; **sorted order** critical for readability.

Grouped Bar: Multiple Series

Compare **multiple series** across categories (2–3 series max)

- Use distinct colors per series
- Include legend with clear labels
- Side-by-side typically better than stacked
- Max 3 groups for readability

```
sns.barplot(data=df, x='region', y='sales', hue='product')
```



Scatter Plot: Bivariate Relationships I

When to Use

Two continuous variables; reveal **correlation**, **clusters**, **outliers**; add regression line to quantify trend.

Key Features:

- **Position**: encodes both X and Y values
- **Regression line**: `sns.regplot()` shows trend
- **Color/size**: encode 3rd or 4th variables
- **Jitter**: reveal overlapping points

```
sns.scatterplot(data=df, x='age', y='income')
```



Scatter Plot: Bivariate Relationships II

When to Use

Two continuous variables; reveal **correlation**, **clusters**, **outliers**; add regression line to quantify trend.

Bubble Plot: Three+ Variables

Scatter + **size** encoding 3rd variable (optional color for 4th)

- **Position**: X–Y relationship
- **Size**: magnitude of 3rd variable
- **Color**: category or continuous 4th dimension

```
sns.scatterplot(data=df, x='gdp', y='life_exp',  
size='population')
```



Heatmap: Correlations

When to Use

Correlation matrices between many variables; **2-D aggregates** (e.g., revenue by region $\times$ product); quickly **spot patterns** and **dependencies**.

For Correlations:

- **Color intensity:** -1 (red, negative) to $+1$ (green, positive)
- **Diagonal:** always 1.0 (self-correlation)
- **Symmetry:** heatmap is **symmetric**
- Remove high-VIF features if possible

```
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
```



Heatmap: 2-D Aggregates

When to Use

Correlation matrices between many variables; **2-D aggregates** (e.g., revenue by region $\times$ product); quickly **spot patterns** and **dependencies**.

For 2-D Aggregates:

- **X-axis:** category 1 (e.g., region)
- **Y-axis:** category 2 (e.g., product)
- **Cell color:** aggregated value (sum, mean, count)
- Find **hot spots** or under-performing cells

```
pivot = df.pivot_table(index='product', columns='region')  
sns.heatmap(pivot)
```



Pair Plot: Exploratory Multivariate Analysis I

When to Use

EDA on datasets with 3–8 variables; visualize **all pairwise relationships** simultaneously; identify **non-linear patterns**, **clusters**, and **outliers**.

Structure:

- **Diagonal:** univariate distribution (histogram or KDE)
- **Off-diagonal:** scatter plot of X vs. Y
- **Color by group:** reveal natural **clusters** or classes
- Grid is **symmetric** and contains redundant information

Limitations:

- Explodes visually with > 10 variables
- Still misses high-dimensional patterns



Pair Plot: Exploratory Multivariate Analysis II

When to Use

EDA on datasets with 3–8 variables; visualize **all pairwise relationships** simultaneously; identify **non-linear patterns**, **clusters**, and **outliers**.

```
sns.pairplot(df, hue='species')
```

Interpretation Tips:

- Look for **linear trends** on scatter plots
- Check for **clusters** of colored points
- Identify **outliers** away from main cloud
- Columns with **uniform distributions** = **weak predictors**
- Strong color separation = good classifier potential



Pie Chart: Part-to-Whole (*Use Sparingly!*) I

When to Use

Part-to-whole relationships with **2–5 slices** as percentages of a total.

Best practice: prefer bar charts for more accurate comparisons.

Pros:

- Intuitive for **percentages** and proportions
- Visually appealing for **executive summaries**
- Good for **2–3 categories maximum**

Cons (Why Avoid):

- **Hard to compare** similar-sized slices (30
- **3-D effects distort** perception severely
- **Bar charts are more accurate** for precise comparisons

```
plt.pie(data, labels=labels, autopct='%1.1f%%')
```



Pie Chart: Part-to-Whole (*Use Sparingly!*) II

Stacked Bar: Better Alternative

Show **composition** of total across multiple **categories**

- Compare totals **and** individual components
- Works for time-series with multiple series
- Multiple categories (unlike pie)
- Easier to read and compare segments

Tips:

- Sort by total for readability
- Max 5 – 6 stacked segments
- Include legend with clear labels

```
df.plot(kind='bar', stacked=True)
```



Line Plot: Trends Over Time I

When to Use

Time-series data; show **trends**, **seasonality**, **anomalies**; compare **multiple series over time**; most effective for **continuous time intervals**.

Design Principles:

- **X-axis**: chronological order (left → right)
- **Y-axis**: starts at 0 (unless showing
- **Multiple lines**: max 3–5 for clarity; use distinct colors
- **Dates**: ISO 8601 format (YYYY-MM-DD)

```
plt.plot(dates, values, marker='o')  
plt.xticks(rotation=45)
```



Line Plot: Trends Over Time

When to Use

Time-series data; show **trends**, **seasonality**, **anomalies**; compare **multiple series over time**; most effective for **continuous time intervals**.

Enhancements:

- **Confidence bands**: fill between upper/lower bounds
- **Markers**: dots at data points (not too dense)
- **Annotations**: label peaks/troughs with dates
- **Moving average**: smooth noise, reveal trend

Common Patterns:

- **Trend**: sustained up or down movement
- **Seasonality**: repeating peaks and valleys
- **Anomaly**: sudden spike or dip
- **Drift**: gradual shift in baseline



Area Chart: Cumulative Volumes Over Time I

When to Use

Show **stacked time-series** with cumulative volumes; emphasize **magnitude** of each series and how **total changes** over time.

Pros:

- Visually appealing; fills space efficiently
- Shows **part-to-whole** relationship over time
- Emphasizes **magnitude** (area $\propto$ value)

Cons:

- Difficult to compare **middle series**
- **Only bottom series** easy to read accurately
- Can be **misleading** if total is not constant

```
df.plot(kind='area', stacked=True, alpha=0.7)
```



Area Chart: Cumulative Volumes Over Time II

When to Use

Show **stacked time-series** with cumulative volumes; emphasize **magnitude** of each series and how **total changes** over time.

Best Practices:

- **Max 4–5 series** (more becomes unreadable)
- Use **semi-transparency** ($\alpha \approx 0.6$) to show overlap
- Order series by **average magnitude** (largest at bottom)
- Always add **legend** with clear labels
- Consider **non-stacked area** for comparing individual series

Non-Stacked Alternative:

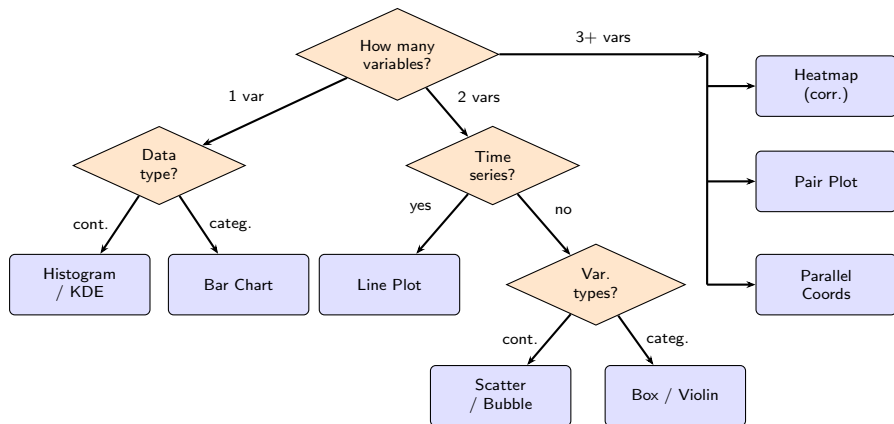
- Better for comparing each series independently
- Use `stacked=False`



Add transparency to see overlaps



Choosing the Right Plot: *Decision Guide*



Outline

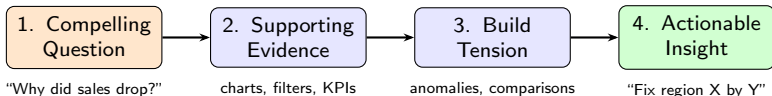
- 1 Data Visualization
- 2 Visual Storytelling and Dashboard Design



Visual Storytelling: Principles

Definition

Combine **data visualization**, **narrative**, and **context** to communicate insights **persuasively** and **memorably** to a specific audience.



Design principles:

- **Color:** purposeful, not decorative; colorblind-safe palettes
- **Annotation:** titles, axis labels, units always
- **Simplicity:** remove chartjunk (gridlines, 3D effects)

Audience alignment:

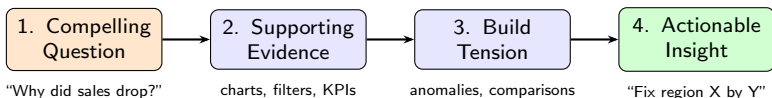
- **Executive:** KPI cards, single headline number
- **Analyst:** drill-down, filters, raw data access
- **Public:** plain language, zero jargon



Visual Storytelling: Principles

Definition

Combine **data visualization**, **narrative**, and **context** to communicate insights **persuasively** and **memorably** to a specific audience.



Design principles:

- **Color:** **purposeful**, not decorative; colorblind-safe palettes
- **Annotation:** titles, axis labels, units **always**
- **Simplicity:** remove chartjunk (gridlines, 3D effects)

Audience alignment:

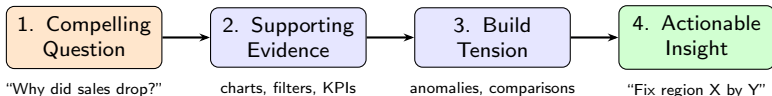
- **Executive:** KPI cards, single headline number
- **Analyst:** drill-down, filters, raw data access
- **Public:** plain language, zero jargon



Visual Storytelling: Principles

Definition

Combine **data visualization**, **narrative**, and **context** to communicate insights **persuasively** and **memorably** to a specific audience.



Design principles:

- **Color:** **purposeful**, not decorative; colorblind-safe palettes
- **Annotation:** titles, axis labels, units **always**
- **Simplicity:** remove chartjunk (gridlines, 3D effects)

Audience alignment:

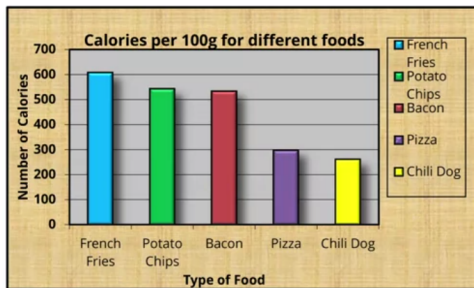
- **Executive:** KPI cards, single **headline number**
- **Analyst:** drill-down, **filters**, raw data access
- **Public:** **plain language**, zero jargon



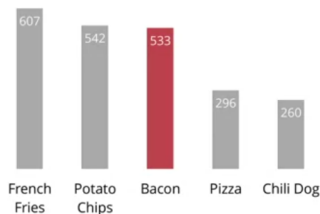
Simplicity is Key: *Data-Ink Ratio*

Data-Ink Ratio (Tufte, 1983)

The proportion of **non-erasable** ink used to represent data compared to the total ink used in the graphic. Higher ratio → more efficient communication.



Calories per 100g



Some color visualization rules

Color Visualization Rules

Use color **purposefully**, not decoratively; ensure colorblind-safe palettes.



Dashboard Design and Best Practices

Dashboard

A **single-screen** visual interface that displays **key metrics, trends, and status** at a glance, enabling **rapid decision-making**.

Design principles:

- Define clear objectives; avoid *information overload*
- Prioritize top-left placement (F-pattern reading)
- Utilize color: large, high-contrast numbers
- Support drill-down and drill-up views
- Refresh: automatic updates, calendar, filters
- Security: role-based permissions, data access (functional)



Dashboard Design and Best Practices

Dashboard

A **single-screen** visual interface that displays **key metrics**, **trends**, and **status** at a glance, enabling **rapid decision-making**.

Design principles:

- Define clear **objectives**; avoid *information overload*
- Prioritize **top-left** placement (F-pattern reading)
- **KPI cards**: large, high-contrast numbers
- Support *drill-down* and **cross-filtering**
- Refresh **automatically** at appropriate cadence
- Separate **governance** (technical) from **storytelling** (functional) dashboards



Dashboard Design and Best Practices

Dashboard

A **single-screen** visual interface that displays **key metrics**, **trends**, and **status** at a glance, enabling **rapid decision-making**.

Design principles:

- Define clear **objectives**; avoid *information overload*
- Prioritize **top-left** placement (F-pattern reading)
- **KPI cards**: large, high-contrast numbers
- Support *drill-down* and **cross-filtering**
- Refresh **automatically** at appropriate cadence
- Separate **governance** (technical) from **storytelling** (functional) dashboards



Dashboard Design and Best Practices

Dashboard

A **single-screen** visual interface that displays **key metrics**, **trends**, and **status** at a glance, enabling **rapid decision-making**.

Design principles:

- Define clear **objectives**; avoid *information overload*
- Prioritize **top-left** placement (F-pattern reading)
- **KPI cards**: large, high-contrast numbers
- Support *drill-down* and **cross-filtering**
- Refresh **automatically** at appropriate cadence
- Separate **governance** (technical) from **storytelling** (functional) dashboards



Dashboard Design and Best Practices

Dashboard

A **single-screen** visual interface that displays **key metrics**, **trends**, and **status** at a glance, enabling **rapid decision-making**.

Design principles:

- Define clear **objectives**; avoid *information overload*
- Prioritize **top-left** placement (F-pattern reading)
- **KPI cards**: large, high-contrast numbers
- Support *drill-down* and **cross-filtering**
- Refresh **automatically** at appropriate cadence
- Separate **governance** (technical) from **storytelling** (functional) dashboards



Dashboard Design and Best Practices

Dashboard

A **single-screen** visual interface that displays **key metrics**, **trends**, and **status** at a glance, enabling **rapid decision-making**.

Design principles:

- Define clear **objectives**; avoid *information overload*
- Prioritize **top-left** placement (F-pattern reading)
- **KPI cards**: large, high-contrast numbers
- Support *drill-down* and **cross-filtering**
- Refresh **automatically** at appropriate cadence
- Separate **governance** (technical) from **storytelling** (functional) dashboards



Dashboard Design and Best Practices

Dashboard

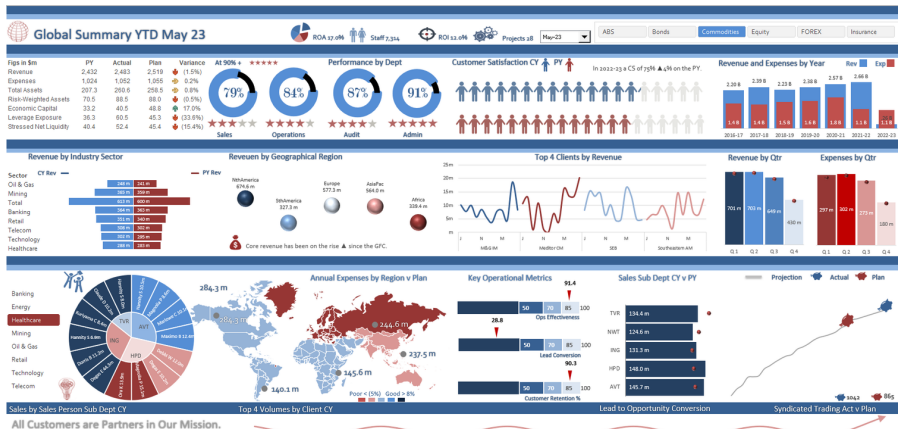
A **single-screen** visual interface that displays **key metrics**, **trends**, and **status** at a glance, enabling **rapid decision-making**.

Design principles:

- Define clear **objectives**; avoid *information overload*
- Prioritize **top-left** placement (F-pattern reading)
- **KPI cards**: large, high-contrast numbers
- Support *drill-down* and **cross-filtering**
- Refresh **automatically** at appropriate cadence
- Separate **governance** (technical) from **storytelling** (functional) dashboards

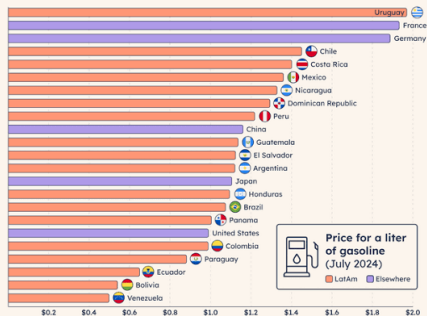


Big Dashboard Example



Insights from a “Big Picture”

The price of gas in Latin America



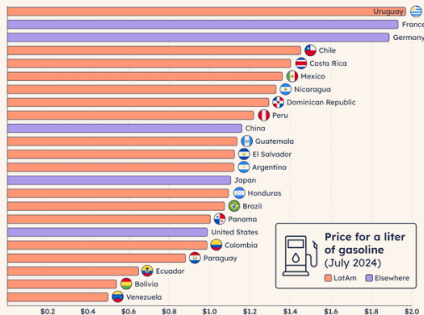
Source: GlobalPetrolPrices.com

Latinometrics



Insights from a “Big Picture”

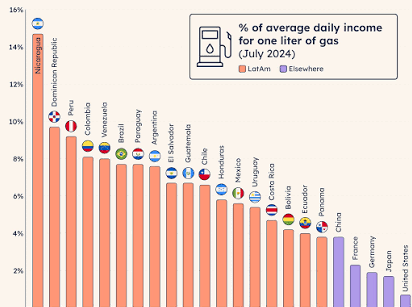
The price of gas in Latin America



Source: GlobalPetrolPrices.com

Latinometrics

How unaffordable is gas in Latin America?



Source: Numbeo, GlobalPetrolPrices.com

Latinometrics



Plotly Dash for Interactive Dashboards

Plotly Dash

A **Python framework** for building **interactive web dashboards** without JavaScript, powered by **Plotly** charts and **Flask** on the backend. The **Gold layer** is the single data source for all Dash applications.

Core components:

- **Layout:** `dcc.Graph()`, `html.Div()`, `dcc.Dropdown()` → reactive UI tree
- **Callbacks:** `@app.callback()` → input → output reactive updates
- **Bootstrap:** `dash-bootstrap-components` or `reactive-grid-layout`

Two dashboard types (Workshop 4):

- **Governance:** KPI cards, null rates, ingestion compliance charts → audience: data engineer
- **Storytelling:** sentiment trends, keyword clouds, temporal patterns → audience: functional user



Plotly Dash for Interactive Dashboards

Plotly Dash

A **Python framework** for building **interactive web dashboards** without JavaScript, powered by **Plotly** charts and **Flask** on the backend. The **Gold layer** is the single data source for all Dash applications.

Core components:

- **Layout:** `dcc.Graph()`,
`html.Div()`,
`dcc.Dropdown()` — reactive
UI tree
- **Callbacks:** `@app.callback()`
— **input** → **output** reactive
updates
- **Bootstrap:**
`dash-bootstrap-components`
for **responsive** grid layout

Two dashboard types (Workshop 4):

- **Governance:** KPI cards, null rates, ingestion compliance charts — audience: data engineer
- **Storytelling:** sentiment trends, keyword clouds, temporal patterns — audience: functional user



Plotly Dash for Interactive Dashboards

Plotly Dash

A **Python framework** for building **interactive web dashboards** without JavaScript, powered by **Plotly** charts and **Flask** on the backend. The **Gold layer** is the single data source for all Dash applications.

Core components:

- **Layout:** `dcc.Graph()`, `html.Div()`, `dcc.Dropdown()` — reactive **UI tree**
- **Callbacks:** `@app.callback()` — **input** → **output** reactive updates
- **Bootstrap:** `dash-bootstrap-components` for **responsive** grid layout

Two dashboard types (Workshop 4):

- **Governance:** KPI cards, null rates, ingestion compliance charts — audience: **data engineer**
- **Storytelling:** sentiment trends, keyword clouds, temporal patterns — audience: **functional user**



Current Challenges in Data Visualization

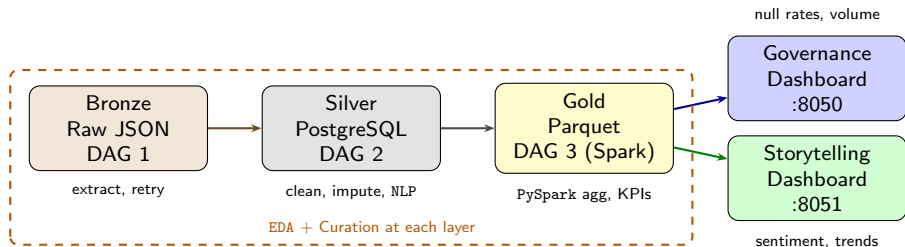
Visualization at Scale

Rendering **billions of data points**, maintaining **responsiveness** with streaming data, and ensuring **accessibility** are engineering challenges that go beyond picking chart types.

- **Scale:** rendering 10^9 points without lag — Deck.gl, WebGL, **data aggregation** before render
- **Dimensionality:** 10+ features in 2D — UMAP, t-SNE, **parallel coordinates**
- **Real-time:** maintaining **responsiveness** with streaming data (Kafka → WebSocket → dashboard)
- **Accessibility:** **colorblind-safe** palettes (viridis, cividis), screen reader support
- **Misleading visuals:** truncated axes, cherry-picked windows, inappropriate chart types — **data literacy** matters



Workshop Connection: Full Pipeline to Dashboard



Key Takeaway

Every slide in this module maps to a **concrete deliverable**: ELT → DAGs, EDA → Silver notebooks, Curation → Silver DAG transforms, Visualization → **Plotly Dash dashboards** (Workshops 3 & 4).



Outline

- 1 Data Visualization
- 2 Visual Storytelling and Dashboard Design



Thanks!

Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/data-analysis-programming>

